

OpenTURNS release highlights

J. Schueller (Phimeca)

User Day #19, June 12th 2026, EDF Lab

AIRBUS



New features since last year in releases:

- v1.26: fall 2025
- v1.27: spring 2026

- 1 Finalized new GPR API migration
- 2 New classes
- 3 Other improvements

Tasks

- Introduced in 1.25 for better hyperparameter optimization, normalization
- Improved scale bounds initialization in 1.26
- Migrated all documentation examples
- Restored Heteroscedastic noise (1.27)
- (1.27)

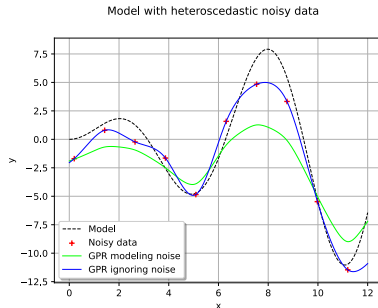
```
fitter = ot.GaussianProcessFitter(x, y, covarianceModel, basis)
fitter.run()
result = fitter.getResult()

algo = ot.GaussianProcessRegression(result)
algo.run()
metamodel = algo.getResult().getMetaModel()
```

Heteroscedastic noise in GPR

- Input-dependent noise modeled via heteroscedastic Gaussian process
- Per-point noise variances set via `setNoise()`

```
import openturns as ot
x = ot.Uniform(0.0, 10.0).getSample(20)
model = ot.SymbolicFunction("x", "x*sin(x)")
y = model(x)
noiseVar = ot.Uniform(0.0, 1.5).getSample(20).as
cov = ot.MaternModel([1.0], 1.5)
fitter = ot.GaussianProcessFitter(x, y, cov...)
fitter.setNoise(noiseVar)
fitter.run()
result = fitter.getResult()
```



Cross-validation for GPR

- Leave-one-out CV residuals computed via closed-form formulas
- No refitting required

LOO residual formula

$$e_{-i} = \frac{[C^{-1}\mathbf{y}]_i}{[C^{-1}]_{ii}} \quad \sigma_{-i}^2 = \frac{1}{[C^{-1}]_{ii}}$$

Ginsbourger, Schärer (2025)

```
fitter = ot.GaussianProcessFitter(x, y, covarianceModel, basis)
fitter.run()

cv = otexp.GaussianProcessRegressionCrossValidation(fitter.getResult())
cv.run()
result = cv.getResult()
r2 = cv.computeR2Score()
stddev = cv.getLeaveOneOutStandardDeviations()
```

openturns.experimental module

- HiGHS – high-performance LP solver
- LinearProblem – framework for solving linear problems
- FiniteOrthogonalFunctionFactory – custom orthonormal bases

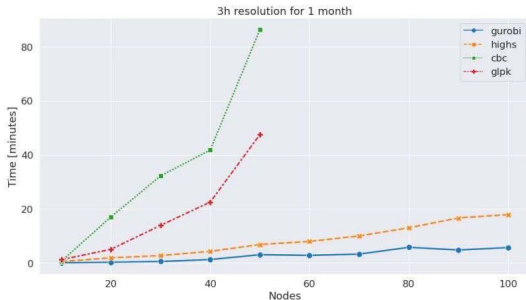
Open-source high-performance LP solver

- Mixed-integer linear programming via HiGHS
- Constraints: $L \leq Ax \leq U$



```
# Maximize x + y + 2z: x + 2y + 3z <= 4,  
cost = [1, 1, 2]  
A = ot.Matrix([[1, 2, 3], [1, 1, 0]])  
LU = ot.Interval([-1e30, 1], [4, 1e30])  
bounds = ot.Interval(3)  
problem = LinearProblem(cost, bounds, A,  
    problem.setVariablesType([BINARY] * 3)  
    problem.setMinimization(False)  
    algo = HiGHS(problem)  
    algo.setStartingPoint([0.0] * 3)  
    algo.run()  
    result = algo.getResult()  
    x = result.getOptimalPoint()  
    y = result.getOptimalValue()
```

Open solvers compared to Gurobi



Custom orthonormal basis

- Build orthonormal basis from arbitrary functions
- Gram-Schmidt orthogonalization wrt any measure

```
dist = distribution = ot.Normal(mean, std, R)
f0 = ot.SymbolicFunction(["x0", "x1"], ["1.0"])
f1 = ot.SymbolicFunction(["x0", "x1"], ["x0"])
f2 = ot.SymbolicFunction(["x0", "x1"], ["-4.0*x0/3.0+5.0*x1/3.0"])
f3 = ot.SymbolicFunction(["x0", "x1"], ["-1.0/sqrt(2.0)+x0^2/sqrt(2)"])
factory = otexp.FiniteOrthogonalFunctionFactory([f0, f1, f2, f3], dist)

prodB = ot.OrthogonalBasis(factory)
bSize = len(factory.getFunctionsCollection())
algo = ot.LeastSquaresExpansion(X, Y, distribution, prodB, bSize)
```

Classes that left the experimental module:

1.26:

- LatentVariableModel
- UniformOrderStatistics
- RankSobolSensitivityAlgo
- ExperimentIntegration
- CubaIntegration
- SamplePartition
- GeneralizedParetoValidation
- FunctionalChaosValidation
- PointToFieldFunctionalChaosAlgorithm

1.27:

- PointConditionalDistribution
- GaussianProcessFitter
- GaussianProcessRegression
- GaussianProcessConditionalCov...
- GaussianProcessRegressionCrossV...
- PosteriorDistribution

Deprecated (1.27):

- KrigingAlgorithm
- KrigingRandomVector
- GeneralLinearModelAlgorithm

In existing classes

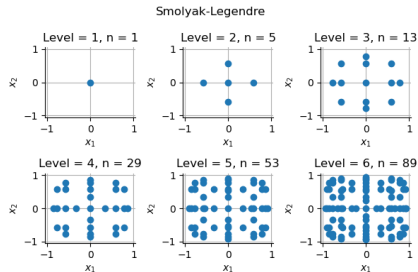
- Smolyak nested tensorization
- MultiStart in FORM
- QuantileConfidence interval coverage
- KFoldSplitter non-interlaced folds
- KDTree radius search
- Pagmo incremental evolution mode
- Documentation improvements

Smolyak nested tensorization

Sparse grid quadrature with nested levels

- Smolyak combination technique via `SmolyakExperiment`
- Efficient merging of duplicate nodes
- Fejér/Clenshaw-Curtis rules support nested levels

```
import openturns as ot
e1 = ot.FejerExperiment(ot.Uniform(0.0, 1.0))
e2 = ot.FejerExperiment(ot.Uniform(0.0, 1.0))
smolyak = ot.SmolyakExperiment([e1, e2], 3)
nodes, weights = smolyak.generateWithWeights()
```



Global design point search

- MultiStart solver accepted by FORM/SORM
- Runs optimizer from multiple starting points
- Avoids local minima in design point computation

```
import openturns as ot
dist = ot.Normal(2)
event = ot.ThresholdEvent(ot.RandomVector(dist), ot.Greater(), 1.0)
start = dist.getSample(10)
optim = ot.MultiStart(ot.Cobyla(), start)
algo = ot.FORM(optim, event)
algo.run()
result = algo.getResult()
```

Actual coverage probability

- Quantile x_α : $P(X \leq x_\alpha) = \alpha$
- Confidence level: $P(x_\alpha \in \text{CI}) \geq \beta$
- New `*WithCoverage` methods return true β

```
import openturns as ot
sample = ot.Normal().getSample(100)
alpha = 0.9
beta = 0.95
algo = ot.QuantileConfidence(alpha, beta)
ci, cov = algo.computeUnilateralConfidenceIntervalWithCoverage(sample)
```

Contiguous fold blocks

- Folds are contiguous (non-interlaced) index ranges
- Each fold: $[0, N/k), [N/k, 2N/k), \dots$
- Optional randomization via `setRandomize(True)`

```
import openturns as ot
sample = ot.Normal().getSample(10)
splitter = ot.KFoldSplitter(sample.getSize(), 5)
for train, test in splitter:
    trainSample = sample[train]
    testSample = sample[test]
```

Range queries with nanoflann

- New queryRadius method for range search
- Returns all points within a given radius
- Optional sorted output by distance

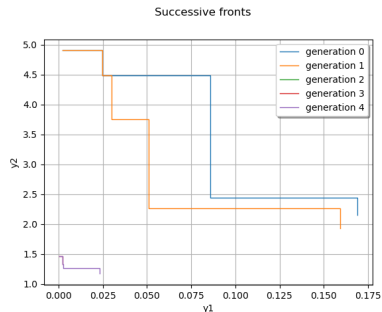
```
import openturns as ot
sample = ot.Normal(3).getSample(100)
tree = ot.KDTree(sample)
indices, dist = tree.queryRadius([0.0] * 3, 0.5, True)
```


Pagmo incremental evolution callback

Per-generation progress reporting

- Evolve one generation at a time instead of all at once
- Continuously report population and Pareto fronts
- Enabled via `setIncrementalEvolution(True)`

```
problem = ot.OptimizationProblem(f)
algo = ot.Pagmo(problem, "nsga2", x0)
algo.setIncrementalEvolution(True)
algo.setProgressCallback(callback)
algo.run()
result = algo.getResult()
```



Documentation improvements

- Improved functions API with graphs
- Improved documentation of Directional sampling classes
- Migrated examples to new GPR API
- New example: pitfalls in polynomial chaos expansion
- Improved Pearson and Spearman theory files



2026 plans

- Functional chaos expansion improvements
- Classification with random forests
- LMG sensitivity indices (dependent variables for linear models)
- Kent + VonMises-Fisher distribution in high dimension
- Distribution factories
- LLM code review

Python channels

- Pip / Conda
- Versions: 3.9+ (abi3)
- OS: Windows, Linux, MacOS
- Architectures: x86_64, arm64 (Linux + MacOS)
- Nightly channel: pypi.anaconda.org/openturns-wheels-nightly

Supported Linux distributions

- Ubuntu 22/24/26
- Debian 11/12/13
- Fedora 43/44
- OpenSUSE 16.0
- Mageia 9
- ArchLinux

... and other various OS/repositories

- FreeBSD
- NixOS
- Spack
- Guix
- Vcpkg
- MSYS2



Join our forum: <https://openturns.discourse.group>



categories ▶

tags ▶

Categories

Latest

Top

+ New Category

✎ New Topic

Category

Topics

Latest

Development

29

Everything about hacking the library.

Announcements

29

Infos on new releases, events, etc

Python usage

89

Get help using openturns from its Python module

Methodology

45

Discussion on the uncertainty quantification methodology in studies

Installation

10



Sphinx gallery issue

Development

1

1d



OpenTURNS Users Day #18, Friday 13, June 2025

Announcements

0

Apr 16



Can an OpenTURNS PythonFunction transmit exceptions from the Python function?

Python usage

4

Mar 13



Is it possible to correlate two distributions in a BayesDistribution?

4

Feb 11

Thank you for your attention!
Any questions?

